

# Algorithms and Data Structures

## Week 3 - Session 2: Tutorial on Queue

### Task: Managing an Online Banking Call Queue

#### Scenario

You are a software engineer at "FinTech Bank" which has a popular online banking helpdesk. Customers call in for support and are placed in a waiting queue until a single, dedicated receptionist (Sarah) can answer their call.

The bank's phone system can only hold a **maximum of 6 callers** in the waiting queue at any time. If the queue is full, new callers receive a busy signal and cannot join.

Sarah handles calls one by one. Once she finishes a call, she connects to the next person in the queue. The system uses a **Circular Queue** to manage this process efficiently, as it reuses the space from callers who have already been served.

#### The Circular Queue Data Structure

A circular queue is the perfect data structure for this scenario because:

- It has a **fixed size** (max 6 callers), preventing system overload.
- It follows a **First-In, First-Out (FIFO)** order, ensuring fairness.
- It efficiently **reuses empty slots** from dequeued elements, avoiding wasted memory.

**Key Components of the Queue:** queue[] of size 6, front, rear, size

#### Key Operations:

- enqueue(caller\_id): Adds a new caller to the end of the queue if it's not full.
- dequeue(): Removes and returns the caller from the front of the queue (to be connected to Sarah).
- isFull(): Checks if the queue is full.
- isEmpty(): Checks if the queue is empty.

### Task 1: Simulate a Day of Calls

Your task is to **simulate the flow of calls** by processing a sequence of events. Each event is either:

- C [Caller\_ID]: A new call comes in from Caller\_ID (e.g., C Alice).
- S: Sarah finishes her current call and connects to the next one in the queue (Serve).

#### Initial State:

- The queue is empty.
- front = -1, rear = -1, size = 0

### Step-by-Step Simulation Exercise

Process the following sequence of events. After **each event**, draw the state of the circular queue, updating front, rear, and size. Also, note any output (e.g., who is being served or if a caller gets a busy signal).

#### Sequence of Events:

|    |                                  | Call queue |   |   |   |   |   |
|----|----------------------------------|------------|---|---|---|---|---|
|    |                                  | 0          | 1 | 2 | 3 | 4 | 5 |
| 1  | C Alice                          |            |   |   |   |   |   |
| 2  | C Bob                            |            |   |   |   |   |   |
| 3  | S (Sarah serves a caller)        |            |   |   |   |   |   |
| 4  | C Charlie                        |            |   |   |   |   |   |
| 5  | C David                          |            |   |   |   |   |   |
| 6  | C Eve                            |            |   |   |   |   |   |
| 7  | S                                |            |   |   |   |   |   |
| 8  | C Frank                          |            |   |   |   |   |   |
| 9  | C Grace                          |            |   |   |   |   |   |
| 10 | C Henry                          |            |   |   |   |   |   |
| 11 | C Isabella // What happens here? |            |   |   |   |   |   |
| 12 | S                                |            |   |   |   |   |   |
| 13 | S                                |            |   |   |   |   |   |
| 14 | C Jack // What happens here?     |            |   |   |   |   |   |
| 15 | S                                |            |   |   |   |   |   |
| 16 | S                                |            |   |   |   |   |   |
| 17 | S                                |            |   |   |   |   |   |

**Task 2: Implement the above Online Banking Call Queue in C#.**